

Hostel Management System

Government OBC & VJNT Boys Hostel, Sangli

Complete Project Explanation & Interview Preparation Guide

Every module, every API route, every important function — explained in plain English, with end-to-end flows, design reasons, known weak spots and likely interview questions.

Prepared for: Harshad Bajirao Khatale

Stack: React · Node.js / Express · MongoDB · Socket.IO · FastAPI · MCP · Groq
LLM

Contents

1. The Project in 60 Seconds	5
Problem it solves	5
Key numbers to remember	5
2. Architecture and Tech Stack	6
Technology choices and why	6
Folder structure	7
How a request travels (big picture)	7
3. Backend Core: Startup, Auth and Utilities	8
3.1 index.js — server bootstrap	8
3.2 Authentication — JWT explained	8
3.3 Utility modules (backend/utils)	9
4. Database Design (Mongoose Models)	10
Design decisions worth explaining	11
5. User, Auth, Profile and Document APIs (userRoutes.js)	12
5.1 Helper functions	12
5.2 Endpoints	12
6. Attendance Module (attendanceRoutes.js)	15
6.1 Business rules	15
6.2 Helper functions	15
6.3 Endpoints	15
6.4 Flow: marking attendance	16
7. Leave Management (leaveRoutes.js)	17
Helper functions	17
Endpoints	17
8. Complaint Box (complaintRoutes.js)	18
9. Real-time Community Chat (Socket.IO)	19
9.1 Server: socket/socketHandler.js	19
9.2 REST: communityRoutes.js	19
9.3 Client side	20
10. Uploads, Notices and Staff	21
10.1 Upload requests (uploadRoutes.js)	21
10.2 Notices (noticeRoutes.js)	21
10.3 Staff (staffRoutes.js)	21
11. AI Assistant — MCP Server + LLM (mcp_server)	22

11.1 What is MCP?	22
11.2 The 8 tools (server.py)	22
11.3 Safety layer — the most important part to explain	22
11.4 LLM orchestration (llm_service.py)	23
11.5 api.py and the admin UI	23
12. Student Portal (frontend)	24
12.1 App shell — App.jsx	24
12.2 Pages	24
12.3 Reusable pieces	25
13. Admin Dashboard (admin)	26
13.1 App.jsx — role-based routing	26
13.2 Pages	26
14. End-to-End Flows (explain these on a whiteboard)	27
Flow A — Student onboarding and login	27
Flow B — Leave to comeback	27
Flow C — Complaint	27
Flow D — Chat message	27
Flow E — AI question	27
15. Security, Performance and Testing	29
Security measures implemented	29
Performance techniques	29
Testing	29
16. Honest Limitations and Future Improvements	31
Feature ideas	31
17. Interview Questions and Answers	32
General / architecture	32
Authentication and security	32
Backend logic	33
Real-time	34
React	34
AI / MCP	35
Behavioural / project questions	35
18. Quick Revision Sheet	36
All API routes at a glance	36
Socket.IO events	36
Environment variables	36

Run commands 36

1. The Project in 60 Seconds

If an interviewer says **"Tell me about your project"**, this is the answer to memorise. Adjust the words, keep the order.

Say it like this: "I built a full-stack Hostel Management System for a government OBC/VJNT boys hostel in Sangli. It has three parts: a student portal and an admin dashboard, both in React, and a Node.js/Express REST API on MongoDB. Students can log in, maintain their profile, upload verification documents to Cloudinary, apply for leave, see their attendance as a yearly heatmap, file complaints with photos, and chat in a real-time community built on Socket.IO. Admins manage students and room allocation with capacity checks, take room-wise attendance inside a fixed 9 PM to midnight window, export styled Excel reports, handle leaves, notices, staff and complaints, and moderate the chat. Security is JWT with role-based access (admin and attendance_taker), bcrypt passwords, and email OTPs through SendGrid for password resets and admin approval. I also added an AI assistant: a Python FastAPI service that exposes safe, read-only MongoDB tools using the Model Context Protocol, and a Groq Llama model decides which tool to call, so the warden can ask questions like 'who was absent yesterday' in plain English."

Problem it solves

- Hostel records (admissions, rooms, attendance, leave, documents) were kept on paper registers, which is slow, error-prone and hard to audit.
- Government hostels need **audit-ready reports** — the Excel export produces daily, monthly (matrix) and yearly attendance sheets.
- Students had no single place to see notices, submit documents, track complaints or apply for leave.
- The warden needed alerts: students who have **not returned from leave**, and students **absent 5+ times in 10 days without leave**.

Key numbers to remember

Item	Value
Sub-projects	4 folders: frontend (student), admin , backend (Express), mcp_server (Python AI)
Ports (dev)	Student 3000, Admin 5173, API + Socket.IO 5000, AI FastAPI 8000
Mongoose models	11: User, Admin, Attendance, LeaveApplication, Complaint, Document, Upload, Notice, Staff, Message, MessageReport
Route modules	8: user, attendance, leave, complaint, community, upload, notice, staff
Roles	student (role user in token), admin , attendance_taker
Rooms	Default 20 rooms × 4 beds, overridable with env ROOM_OVERVIEW (e.g. 1:4,2:6)
Attendance window	21:00 – 24:00 IST, only for today's date
OTP	6 digits, SHA-256 hashed, valid 5 minutes
JWT expiry	7 days
Upload limits	5 MB (profile photo / documents / staff), 10 MB (complaints, upload requests)
AI tools	8 FastMCP tools; max 50 docs per query; max 5 LLM↔tool rounds

2. Architecture and Tech Stack

The system is a **decoupled client–server architecture**: two independent React single-page apps talk to one Express API over HTTP (JSON) and WebSocket. A separate Python service provides the AI assistant. All persistent data lives in MongoDB Atlas; files live in Cloudinary.

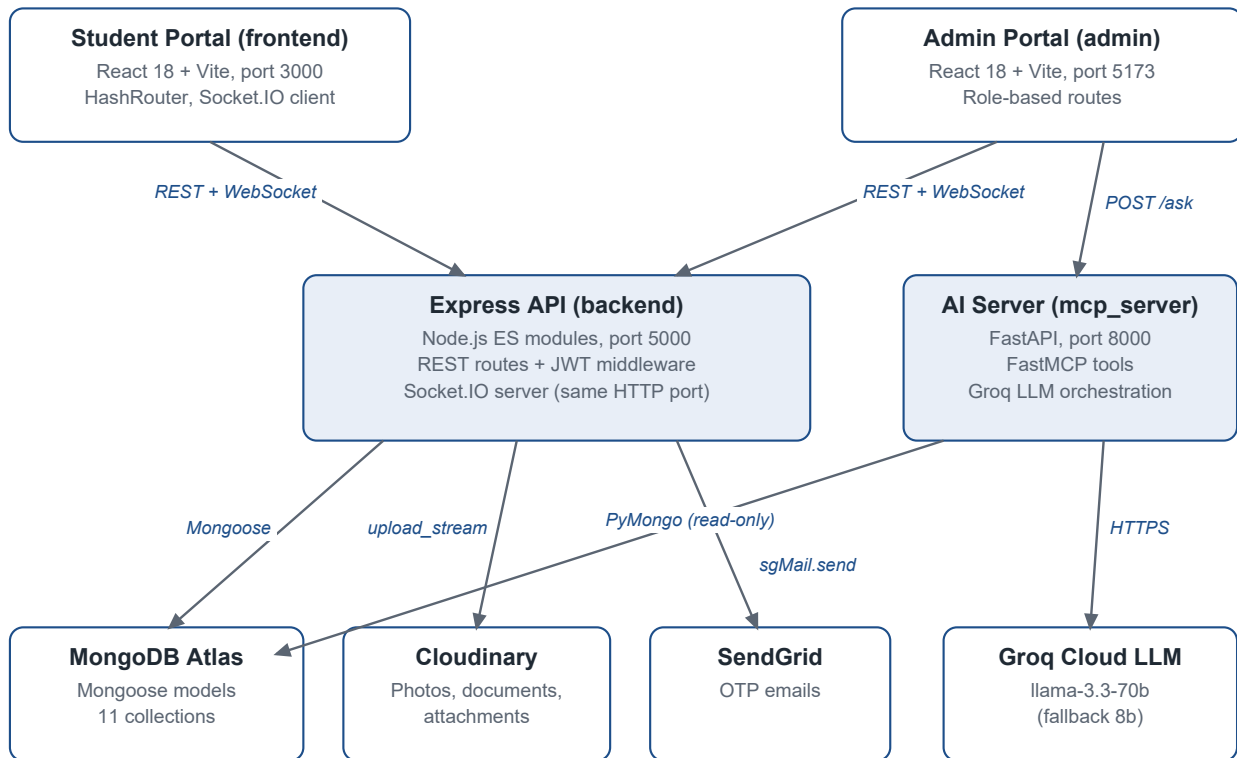


Figure 1 – System architecture

Technology choices and why

Layer	Technology	Why this choice (say this in the interview)
UI	React 18 + Vite	Component-based UI, fast dev server with hot reload, small production bundles.
Routing	React Router v6 with HashRouter	URLs look like <code>/#/profile</code> . Works on static hosting without server rewrite rules (refresh never gives 404).
Icons / images	react-icons, browser-image-compression	Images are compressed in the browser (max ~0.8 MB, 2400 px) before upload, saving bandwidth and Cloudinary storage.
API	Node.js + Express 4 (ES modules)	Same language as the frontend, huge ecosystem, simple middleware model for auth and file upload.
Database	MongoDB + Mongoose 7	Flexible documents fit student profiles; attendance is naturally one document per day with an embedded students array.
Auth	jsonwebtoken + bcryptjs	Stateless JWT means no server session store; bcrypt (10 salt rounds) for password hashing.
Files	Multer (memory) + Cloudinary + streamifier	File stays in RAM as a Buffer and is streamed straight to Cloudinary; no temp files on the server.
Email	SendGrid	Reliable transactional email for OTPs.

Layer	Technology	Why this choice (say this in the interview)
Reports	ExcelJS	Generates real .xlsx with colours, merged title cells, frozen panes and auto-filter.
Real-time	Socket.IO 4	Rooms per channel, automatic reconnection, falls back to long-polling if WebSocket is blocked.
AI	Python, FastAPI, FastMCP, PyMongo, Groq API	MCP is a standard way to expose tools to LLMs; Groq gives fast Llama inference; FastAPI gives a quick REST wrapper.

Folder structure

```

hostel_management_system/
├── backend/                                Express REST API + Socket.IO
│   ├── index.js                          app bootstrap, routes, DB connect, port fallback
│   ├── middleware/authMiddleware.js      authMiddleware, optionalAuth, adminAuth
│   ├── models/                          11 Mongoose schemas
│   ├── routes/                          8 route modules (user, attendance, leave, ...)
│   ├── socket/socketHandler.js          Socket.IO auth, rooms, typing, presence
│   ├── utils/                          passwordOtp, passwordReset, roomUtils
│   └── tests/                          node:test unit tests (7 files)
├── frontend/                             Student & public portal (React)
│   └── src/ pages/ components/ components/sections/ components/community/ utils/ hooks/ css/
├── admin/                                Admin dashboard (React)
│   └── src/ pages/ components/ (Sidebar, HostelNexusWidget) utils/ css/
├── mcp_server/                           AI assistant (Python)
│   ├── server.py                       FastMCP tools + safety layer over MongoDB
│   ├── llm_service.py                  Groq LLM <-> tool-calling loop, memory, fallback
│   ├── api.py                         FastAPI: /ask, /collections, /health
│   └── test_suite.py                   manual integration checks

```

How a request travels (big picture)

1. User clicks a button in React → a `fetch()` call goes to `/api/...` with header `Authorization: Bearer <JWT>`.
2. In development, **Vite's proxy** forwards `/api` and `/socket.io` to `localhost:5000`, so there is no CORS problem. In production the app uses `VITE_API_BASE_URL`.
3. Express parses JSON (`express.json()`) or multipart (Multer), runs the auth middleware, then the route handler.
4. The handler validates input, queries MongoDB through Mongoose, optionally uploads to Cloudinary / sends an email / emits a Socket.IO event.
5. A JSON response goes back; React updates state and re-renders.

3. Backend Core: Startup, Auth and Utilities

3.1 index.js — server bootstrap

`backend/index.js` is the entry point (`npm run dev` uses nodemon, `npm start` uses node).

- Loads `.env` with `import 'dotenv/config'` (must be the first import so every module sees the variables).
- Creates `app = express()` and wraps it in `http.createServer(app)` — this is needed because **Socket.IO attaches to the same HTTP server**, so REST and WebSocket share port 5000.
- `initSocket(httpServer)` sets up real-time chat.
- Warns if `ADMIN_USERNAME` / `ADMIN_PASSWORD` are missing.
- Global middleware: `cors()`, `express.json()`, `express.urlencoded()`, and `express.static` for `/uploads` (local fallback files).
- Mounts routers: `/api/admin` and `/api` → `userRoutes` (mounted twice, so `/api/user/login` and `/api/admin/login` both resolve), `/api/notifications`, `/api/uploads`, `/api/staff`, `/api/attendance`, `/api/leaves`, `/api/community`, `/api/complaints`.
- `GET /` returns a health message.

Functions in index.js

Function	What it does
<code>ensureUsernameIndex()</code>	Creates a unique index on <code>users.username</code> so two students can never share a username, even under a race condition.
<code>ensureDocumentIndexes()</code>	Drops an old <code>email_1</code> index (legacy) and creates a unique index on <code>documents.userId</code> → exactly one Document record per student.
<code>startServer(port, attempt)</code>	Calls <code>httpServer.listen(port)</code> . If the port is busy (<code>EADDRINUSE</code>) it retries on <code>port+1</code> , up to 5 times. Any other error exits the process.
<code>mongoose.connect(...)</code>	Connects to MongoDB first; only after a successful connection does it build the indexes and start listening, so the API never serves requests without a database.

3.2 Authentication — JWT explained

After a correct username + password, the server signs a token with `jwt.sign(payload, JWT_SECRET, { expiresIn: '7d' })`. The payload is small:

```
{ userId: "<Mongo_id>", username: "rahul", role: "user" | "admin" | "attendance_taker" }
```

The client stores this token in `localStorage` (`hostelToken` for students, `adminToken` for admins) and sends it on every request. The server never stores sessions — it just verifies the signature with `jwt.verify`. That is what **stateless** means.

Middleware functions (middleware/authMiddleware.js)

Middleware	Behaviour
<code>getTokenFromHeader(req)</code>	Reads <code>Authorization</code> , requires the <code>Bearer</code> prefix, returns the token or null.
<code>authMiddleware</code> (default export)	Any logged-in user. No token → 401 "Missing authorization header"; bad/expired → 401. On success sets <code>req.user = decoded payload</code> .
<code>optionalAuth</code>	Never blocks. If a valid token is present, sets <code>req.user</code> ; else continues anonymously. Used for reading community channels.

Middleware	Behaviour
<code>adminAuth</code>	Token must decode to role <code>admin</code> (or legacy username <code>admin</code> / id <code>hardcoded-admin</code>). Non-admin → 403 (authenticated but not allowed).

Why it matters: 401 means "I don't know who you are" (missing/invalid token). 403 means "I know who you are, but you are not allowed" (wrong role).

Stronger admin check inside `userRoutes.js`

`userRoutes` defines its own `adminAuth` and `attendanceAuth` that go one step further: after verifying the JWT they **load the Admin document from MongoDB** and reject the request if `isActive` is false or `status !== 'active'`. So a disabled admin is locked out immediately even though their token has not expired. `req.user.role` is also refreshed from the database, not trusted from the token.

Role model

Role	Where it comes from	Can do
Student (<code>user</code>)	User collection; created only by admin via Add User	Own profile, documents, leave, complaints, chat, upload submissions
<code>admin</code>	Admin collection; self-registration approved by root-admin OTP	Everything
<code>attendance_taker</code>	Admin collection with role <code>attendance_taker</code>	Only Attendance and Attendance Visuals pages / endpoints

3.3 Utility modules (backend/utills)

`passwordOtp.js`

Function	Explanation
<code>generateOtp()</code>	Returns a 6-digit string: <code>100000 + Math.random() * 900000</code> .
<code>hashOtp(otp)</code>	SHA-256 hash via Node <code>crypto</code> . The plain OTP is only emailed, never stored .
<code>createOtpPayload()</code>	Returns <code>{ otp, otpHash, expiresAt: now + 5 min }</code> . The route stores only hash + expiry and emails the otp.
<code>verifyOtp(entry, otp)</code>	False if entry missing, expired, or hash mismatch; true otherwise.
<code>createOtpEntry()</code>	Helper that creates an entry without exposing the OTP (used in tests).

`passwordReset.js` — `validatePasswordResetPayload()`

Pure validation function: username, OTP and new password required; password ≥ 6 characters; `newPassword === confirmPassword`. Returns `{ ok, message }`. Keeping it pure makes it easy to unit test.

`roomUtils.js`

Function	Explanation
<code>normalizeRoomNumber(v)</code>	Converts to string, trims, removes all whitespace ("A 10" → "A10"). Empty/null → "".
<code>roomNumberMatches(a, b)</code>	Compares two room numbers after normalising.
<code>parseRoomOverview(str)</code>	Parses <code>ROOM_OVERVIEW</code> like "1:4,2:4,3:6" into <code>[{roomNumber: '1', capacity: 4}, ...]</code> . Defaults to 20 rooms × 4. Skips duplicates; invalid capacity → 4.

4. Database Design (Mongoose Models)

MongoDB stores **documents** (JSON-like). Each model below is a Mongoose schema → a collection. Relationships use **ObjectId** references plus **populate()**, while attendance uses **embedding** (students array inside a day document).

Model / collection	Important fields	Notes
User (students)	username (unique), password (bcrypt), rollNumber, email, phone, college_name, stream, department, year, roomNumber, photoUrl, fullName, address, village, taluka, district, mobileNumber, fathersMobileNumber, aadhaarNumber, caste, casteCategory, BankName, bankBranch, accountNumber, ifscCode, admissionDate, createdAt	Profile + scholarship/bank data. Password is removed from every response by <code>getPublicUserData()</code> .
Admin	username (unique, lowercase), password, email (unique), phone, role: admin attendance_taker, status: pending_verification active disabled, isActive, lastLoginAt	Separate collection from students — admin and student logins never mix.
Attendance	date 'YYYY-MM-DD' (unique index), students: [{username, roomNumber, status: Present Absent}], markedBy, firstSavedAt	One document per day. Each room save replaces only that room's entries inside the array.
LeaveApplication	userId → User, username, fullName, reason, startDate, endDate (strings), mobileNumber, attachmentUrl, status: Pending Approved Rejected, comebackMarked, comebackDate, comebackReminderSent, notificationCount, adminNote	Tracks the full leave life-cycle including return ("comeback").
Complaint	title (≤200), description, category (8 enums), priority: Low Medium High Urgent, imageUrl, status: Pending In Progress Resolved Rejected, student snapshot fields, userId, adminResponse {response, respondedBy, respondedAt}, resolvedAt	Student name/room are snapshotted so the complaint remains readable even if the user changes.
Document	userId (unique) → User, email, and 8 URL fields: aadharCard, casteCertificate, incomeCertificate, domicileCertificate, collegeAdmissionReceipt, bonafideCertificate, casteValidityCertificate, previousYearMarksheet	One record per student holding Cloudinary links to verification documents.
Upload	title, description, dueDate, requestedBy, submissions: [{userId, username, fileUrl, publicId, uploadedAt}]	Admin-created "please submit X" request; students submit into the embedded array.
Notice	title (≤200), content, severity: low medium high, timestamps	Notice board.
Staff	name, position, email, phone, imageUrl, publicId	Shown on the public home page.
Message	channel: announcement general (indexed), senderId (refPath → senderModel User Admin), content (≤2000), replyTo → Message, reactions [{emoji, users[]}], isEdited, isDeleted, deletedAt, deletedBy, deletedByRole: owner admin	Compound index <code>{channel:1, createdAt:-1}</code> makes "latest messages in channel" fast.
MessageReport	messageId → Message, reportedBy → User, reason (6 enums), description, status: Pending Reviewed Dismissed Action Taken, reviewedBy, reviewedAt	Moderation queue.

Design decisions worth explaining

- **Attendance embedded per day** instead of one row per student per day: one read gives a whole day for all rooms; the unique index on `date` prevents duplicate days. Trade-off: the document grows with number of students (fine for a ~80 bed hostel, far below the 16 MB limit).
- **Dates stored as 'YYYY-MM-DD' strings** in Attendance and Leave: string comparison works for ranges (`'2026-09-01' <= d <= '2026-09-30'`) and avoids timezone shifts that happen with JavaScript `Date` at midnight.
- **refPath on Message.senderId**: a message sender can be either a User or an Admin, so Mongoose decides the model dynamically from `senderModel`.
- **Soft delete** for messages (`isDeleted` flag) keeps reply threads and reports intact, and gives an audit trail.
- `timestamps: true` adds `createdAt` / `updatedAt` automatically.

5. User, Auth, Profile and Document APIs (userRoutes.js)

The biggest route file (~1,230 lines). It is mounted at both `/api/admin` and `/api`. Multer is configured with **memoryStorage** and a **5 MB limit**. Four in-memory **Maps** hold pending OTPs: `passwordOtpStore`, `passwordResetOtpStore`, `adminRegistrationOtpStore`, `adminDeleteOtpStore`.

5.1 Helper functions

Helper	Purpose
<code>configureSendGrid() / sendPasswordOtpEmail(to, otp)</code>	Sets the API key and sends the OTP email (text + HTML) to the user.
<code>getRootAdminEmail() / sendAdminOtpEmail(otp, purpose)</code>	Sends admin registration / deletion OTPs to the root admin email (<code>ADMIN_ROOT_EMAIL</code>), not to the requester. This makes the root admin an approval gate.
<code>configureCloudinary()</code>	Validates all three Cloudinary env vars and rejects placeholder values like <code>your_...</code> ; throws a clear error if missing.
<code>uploadStudentPhoto(buffer, name, mime)</code>	Streams a Buffer into Cloudinary folder <code>hms_students</code> with <code>quality:auto</code> , <code>fetch_format:auto</code> ; resolves the <code>secure_url</code> .
<code>uploadDocumentToCloudinary(...)</code>	Same for folder <code>hms_documents</code> with <code>resource_type: 'auto'</code> so PDFs and images both work.
<code>getCloudinaryPublicId(url)</code>	Regex <code>/\/upload\/(?:v\d+\/)?(.+)\.[^.]*/</code> extracts <code>folder/file</code> from a URL — needed to delete the old asset.
<code>deleteCloudinaryImage(url)</code>	Deletes an old asset when a photo/document is replaced, so storage does not fill with orphans.
<code>getPublicUserData(user)</code>	<code>toObject()</code> then deletes <code>password</code> , <code>tempPassword</code> , <code>__v</code> → safe to send to client.
<code>formatDate(date)</code>	Local date → 'YYYY-MM-DD'.
<code>normalizeDocumentField(name)</code>	Maps form field names (<code>aadharCard</code> or <code>aadharCardUrl</code>) to the schema field name.
<code>findAuthenticatedUser(tokenUser)</code>	Finds the student by <code>userId</code> (if valid ObjectId) else by username; excludes password.

5.2 Endpoints

Method & path	Guard	What happens
POST <code>/add-user</code>	<code>adminAuth</code>	Admin creates a student. Requires username, password, roomNumber. Validates email regex, checks username uniqueness (409), checks the room exists in <code>ROOM_OVERVIEW</code> and current occupancy < capacity ("Capacity full"), hashes password with <code>bcrypt(10)</code> , saves. Returns the temp password so the admin can hand it over.
PUT <code>/users/:id/email</code>	<code>adminAuth</code>	Update student email with regex validation.
PUT <code>/users/:id/rollnumber</code>	<code>adminAuth</code>	Update roll number.
PUT <code>/users/:id/room</code>	<code>adminAuth</code>	Room change with capacity check (excluding the student himself). If the student already has attendance history and <code>confirmAttendanceChange</code> is not true, returns 409 with requiresConfirmation — the UI then asks the admin to confirm.

Method & path	Guard	What happens
POST <code>/register</code>	public	Admin/attendance-taker sign-up step 1. Validates fields, role must be <code>admin</code> or <code>attendance_taker</code> , checks duplicates, stores the pending data with an OTP hash in <code>adminRegistrationOtpStore</code> , emails OTP to root admin.
POST <code>/register/verify</code>	public	Step 2. Verifies OTP, hashes password, creates the Admin (active), deletes the pending entry, returns a JWT.
POST <code>/login</code> (→ <code>/api/admin/login</code>)	public	Admin login. Lower-cases username, rejects inactive accounts (403), <code>bcrypt.compare</code> , updates <code>lastLoginAt</code> , signs a 7-day JWT with role.
POST <code>/user/login</code>	public	Student login; JWT role <code>user</code> . Same generic "Invalid username or password" for both wrong user and wrong password (prevents username enumeration).
POST <code>/user/forgot-password</code>	public	Finds student, requires a registered email, stores OTP hash, emails OTP.
POST <code>/user/reset-password</code>	public	Validates payload, verifies OTP, <code>bcrypt</code> -hashes new password, deletes OTP (single use).
POST <code>/forgot-password</code> , <code>/reset-password</code>	public	Same two-step reset for admin accounts.
GET <code>/admin-users</code>	adminAuth	Lists all admin accounts without passwords.
POST <code>/admin-users/:id/request-delete-otp</code>	adminAuth	Cannot target yourself. Emails deletion OTP to root admin.
DELETE <code>/admin-users/:id</code>	adminAuth	Deletes an admin only with the valid OTP.
GET <code>/profile</code>	auth	Returns the logged-in student's profile.
POST <code>/profile/password/otp</code>	auth	Change-password step 1: new password ≥ 6 chars, stores OTP + pending password, emails OTP.
POST <code>/profile/password/verify</code>	auth	Step 2: verifies OTP, hashes and saves password.
POST <code>/profile</code>	auth + multer <code>studentPhoto</code>	Whitelists 21 profile fields (so a student cannot set <code>roomNumber</code> or <code>password</code> here), parses <code>admissionDate</code> , replaces photo in Cloudinary (deletes old one), <code>findByIdAndUpdate</code> with <code>\$set</code> .
GET <code>/users</code>	adminAuth	All students + computed <code>presentDaysSinceCreated</code> and <code>totalAttendanceDaysSinceCreated</code> by scanning attendance documents into a Map keyed by username.
GET <code>/rooms-overview</code>	adminAuth	For each configured room: capacity, occupancy, available, status <code>alloted/free</code> ; plus totals.
GET <code>/users/counts?groupBy=casteCategory caste</code>	adminAuth	MongoDB aggregation <code>\$group</code> + <code>\$sort</code> for the dashboard chart. Declared <code>before /users/:id</code> so Express does not treat "counts" as an id.
GET <code>/users/:id</code>	auth	Single student.
DELETE <code>/users/:id</code>	auth	Deletes student, their Cloudinary photo, pulls them from all Attendance docs (<code>\$pull</code>), removes their upload submissions, deletes their Document record.
GET <code>/documents</code>	auth	Returns (or lazily creates) the student's Document record; handles duplicate-key race (error 11000) by re-reading.
POST <code>/documents</code>	auth + <code>upload.any()</code>	For each uploaded file: map field name, delete previous Cloudinary asset, upload new one, save URL.
GET <code>/users/:id/documents</code>	auth	Admin view of a student's documents.

Say it like this: "Room allocation is capacity-checked on the server with `countDocuments({roomNumber})` against the capacity parsed from an environment variable, so the frontend cannot over-fill a room. Changing a room when attendance history exists needs an explicit confirmation, because it affects future attendance grouping."

6. Attendance Module (attendanceRoutes.js)

This is the most business-heavy module (~1,225 lines). It uses its own `adminAuth` that allows role **admin** or **attendance_taker**.

6.1 Business rules

- Attendance is taken **room by room** (rooms 1–20, constant `ROOM_COUNT`).
- It can be marked **only for today** and **only between 21:00 and 24:00 IST** (`isAttendanceWindowOpen`: minutes-of-day ≥ 1260 and < 1440 , computed in `Asia/Kolkata` time zone).
- Outside the window an existing day's record is **locked** (`isRecordLocked`), so nobody can change history.
- Students with **approved leave** covering the date are forced to Absent, even if ticked present — leave is handled separately.
- All data is saved into the single Attendance document for that date; saving room 5 only replaces room 5's entries.

6.2 Helper functions

Function	Explanation
<code>getFormattedDate(d)</code>	Date $\rightarrow$ 'YYYY-MM-DD' in server local time.
<code>normalizeDateString,</code> <code>normalizeDate(date)</code>	Validate the <code>^\d{4}-\d{2}-\d{2}\$</code> pattern; default to today.
<code>normalizeRoomNumber(r)</code>	Must be an integer 1–20, else null $\rightarrow$ 400.
<code>isAttendanceWindowOpen()</code>	Converts now to IST via <code>toLocaleString('en-US', {timeZone: 'Asia/Kolkata'})</code> ; true between 9 PM and midnight.
<code>getDatesInRange(start, end)</code>	Array of every date between two dates (inclusive).
<code>getRecentDateStrings(days)</code>	Last N dates ending today (for alerts).
<code>resolveAbsentsDateRange({date, year, month})</code>	Exported & unit-tested. A single date $\rightarrow$ one-day range; otherwise whole month (validates year 2000–2100, month 1–12).
<code>isDateCoveredByLeave(leave, date)</code>	Approved leave with $\text{start} \leq \text{date} \leq \text{end}$, and not after the student's comeback date.
<code>getUnapprovedAbsenceRows(start, end)</code>	Loads attendance, users and approved leaves in parallel (<code>Promise.all</code>), builds Maps for O(1) lookup, collects Absent entries not covered by leave , de-duplicates dates, sorts by absentCount desc.
<code>buildAttendanceMatrix(students, records, dates)</code>	Builds a student $\times$ date grid of 'P'/'A', plus presentDays, markedDays, absentDays and 'present/marked' text. Used by range report and monthly Excel.
<code>isRecordLocked(firstSavedAt)</code>	Locked when a record exists and the window is closed.

6.3 Endpoints

Endpoint	Explanation
GET <code>/summary?date=</code>	Dashboard for a day: for each of 20 rooms — capacity (students assigned), present, absent, marked, Completed/Pending, locked. Totals + overall percentage.
GET <code>/room/:roomNumber?date=</code>	Students of that room with their current status (<code>isMarked</code> tells whether it was actually saved), lock state.

Endpoint	Explanation
POST <code>/room/:roomNumber</code>	Save attendance. Body <code>{date, presentUserIds}</code> . Checks: valid room/date, date is today (403), window open (403), not locked, room not empty (404), every present id belongs to this room (400). Loads approved leaves and forces those students Absent. Merges with other rooms' entries and saves with <code>findOneAndUpdate(..., {upsert:true})</code> ; <code>\$setOnInsert</code> sets <code>firstSavedAt</code> only the first time.
GET <code>/student/my-attendance?year=</code>	Student's own view: today's status, monthly % and yearly %, a 365/366-day heatmap array (status Present/Absent/NotMarked/Future with <code>dayOfWeek</code> and <code>month</code>), and last 30 records. Handles leap years.
GET <code>/student/:id/attendance</code>	Same data for admin viewing any student.
GET <code>/visuals?year=&month=</code>	Present-count per day of month for the line chart.
GET <code>/absence-alerts?threshold=5&days=10</code>	Students absent $\geq$ threshold times in the last N days without approved leave → dashboard notifications.
GET <code>/absents?date=</code> or <code>?year=&month=</code>	List of unapproved absences for a day or month.
POST <code>/range-report</code>	Custom date range (max 366 days) → matrix + per-room totals + attendance percentage per student.
GET <code>/export?type=daily monthly yearly</code>	Streams an Excel file . Monthly: matrix sheet (Roll No, Student, Room, Name, Email, Phone, one column per day, Total), dark-blue merged title, frozen first 6 columns and header rows, auto-filter, green P / red A. Daily: flat list. Yearly: grouped by month → date sections. Sets <code>Content-Type</code> for <code>xlsx</code> and <code>Content-Disposition: attachment</code> and writes with <code>workbook.xlsx.write(res)</code> .

6.4 Flow: marking attendance

- Attendance taker opens Admin → Attendance. `fetchSummary(date)` calls `/summary` and shows 20 room cards (Completed / Pending / Locked).
- Clicking a room calls `openRoomModal()` → GET `/room/:n`. Students appear with checkboxes; `handleSelectAll(true/false)` toggles everyone.
- `handleSaveAttendance()` POSTs the ids of ticked students.
- Server validates time window and room membership, applies leave rules and upserts the day document.
- Summary refreshes; the room turns Completed. After midnight the whole day becomes read-only.
- Reports: `handleExport(type)` downloads the Excel; Attendance Visuals page shows the monthly line chart (hand-drawn SVG polyline), absentees list and custom range report.

Be ready for this: The error message in code says "8:00 PM to 12:00 AM" but the actual check is 1260 minutes = **9:00 PM**. If asked, say the window is configurable business logic and you would move it to an environment variable and fix the message.

7. Leave Management (leaveRoutes.js)

Handles the full life-cycle: apply → (approved) → student is away → comeback marked → old records cleaned. Attachments go to Cloudinary folder `hostel-leaves`, **falling back to local disk** (`backend/uploads`) if Cloudinary is not configured or fails.

Helper functions

Function	Explanation
<code>parseLeaveDate(v)</code>	'YYYY-MM-DD' → <code>Date.UTC(...)</code> (UTC avoids timezone off-by-one).
<code>getDatesInRange</code>	All dates of the leave using <code>setUTCDate</code> .
<code>getLeaveAttendanceDates(leave)</code>	Leave dates, but cut off at the comeback date if the student returned early.
<code>isLeaveActiveOnDate(leave, d)</code>	Approved, inside range, and before comeback.
<code>getExpectedReturnDate(leave)</code>	End date (or planned comeback date) if not yet returned.
<code>isValidDateRange(s, e)</code>	Both valid and start ≤ end.
<code>hasOverlappingActiveLeave(userId, s, e, excludeId)</code>	Classic interval-overlap query: <code>startDate <= e AND endDate >= s</code> among Pending/Approved leaves → prevents double booking.
<code>markApprovedLeaveAttendance(leave)</code>	For every leave date, upserts the Attendance day and sets this student Absent with <code>markedBy: 'leave-auto'</code> .
<code>syncAttendanceAfterComeback(leave)</code>	After early return, removes the auto-generated absent entries from comeback date onward.
<code>cleanupExpiredLeaveApplications()</code>	Deletes leaves 5 days after their end date — except approved leaves where the student has not yet come back (those must stay visible). Runs on each leave request and every hour via <code>setInterval</code> .
<code>uploadToCloudinary / saveLocalAttachment</code>	Upload with automatic local fallback.

Endpoints

Endpoint	Explanation
POST <code>/submit</code> (auth, multer <code>attachment</code>)	Validates dates, attachment mandatory , rejects overlapping leave (409), uploads proof, creates the leave with status Approved (auto-approved in the current version) and marks attendance Absent for those dates.
GET <code>/my</code> (auth)	Student's leave history, newest first.
GET <code>/all</code>	All leaves with <code>populate('userId', 'username fullName roomNumber')</code> for the admin Leave Management page.
PATCH <code>/:id/status</code>	Admin sets Pending/Approved/Rejected with a note. On approve: overlap check + mark attendance. On non-approve: resets all comeback fields.
PATCH <code>/:id/comeback</code>	Marks the student as returned on a date (≥ start date), then cleans future auto-absents.
GET <code>/notifications</code>	Overdue alert: approved leaves whose end/comeback date has passed but comeback is not marked → "Student has not returned from leave". Increments <code>notificationCount</code> . Shown on the admin dashboard together with absence alerts.

Say it like this: "Leave and attendance are linked: approving a leave automatically writes Absent entries marked 'leave-auto', and the absence-alert query excludes dates covered by approved leave, so the warden only sees truly unexplained absences."

9. Real-time Community Chat (Socket.IO)

9.1 Server: socket/socketHandler.js

Part	Explanation
<code>initSocket(httpServer)</code>	Creates new <code>Server(httpServer, {cors})</code> and stores it in a module variable so routes can use <code>getIO()</code> .
<code>io.use(...)</code> auth middleware	Runs during the handshake . Reads token from <code>socket.handshake.auth.token</code> , verifies JWT. Admin → builds an admin identity; student → loads User from DB. Any failure → connection refused. So only logged-in users can open a socket.
Presence (<code>onlineUsers</code> Map)	<code>userId</code> → Set of socket ids (a user may have several tabs). First socket → broadcast <code>user:online</code> ; last socket closed → <code>user:offline</code> .
<code>community:join</code> / <code>community:leave</code>	Adds / removes the socket to a room named after the channel ('general' or 'announcement').
<code>typing:start</code> / <code>typing:stop</code>	Relayed with <code>socket.to(room).emit</code> → everyone in the room except the sender sees "X is typing...".
<code>getIO()</code>	Lets REST routes emit events after saving to DB.

9.2 REST: communityRoutes.js

Two fixed channels (`STATIC_CHANNELS`): **Announcements** (only admin can post) and **General**. Messages are **written through REST and broadcast through Socket.IO** — that way every message is validated and saved first, and history can be loaded normally.

Endpoint	Explanation
GET <code>/channels</code> (optionalAuth)	Returns the two channels.
GET <code>/channels/:id/messages?before=&limit=</code>	Newest-first query with limit (≤ 100), then reversed to chronological. <code>before</code> gives cursor pagination (load older). Populates sender, reply-to message + its sender, reaction users.
POST <code>/channels/:id/messages</code>	Validates non-empty, ≤ 2000 chars; server-side rule: non-admin cannot post in Announcements (403) . Validates replyTo id. Saves, populates, emits <code>message:receive</code> to the channel room.
PUT <code>/messages/:id</code>	Edit — only the sender or an admin; sets <code>isEdited</code> ; emits <code>message:update</code> .
DELETE <code>/messages/:id</code>	Soft delete — sender or admin. Replaces content with a notice ("removed by Admin..." or "deleted by owner"); emits <code>message:delete</code> .
POST <code>/messages/:id/react</code>	Toggle logic: emoji not present → add; user already reacted → remove (and drop emoji if no users left); else add user. Emits <code>message:react</code> .
POST <code>/messages/:id/report</code>	Creates a MessageReport with reason + description.
GET / PUT <code>/admin/reports</code>	Admin moderation: list reports; update status and optionally <code>action: 'delete_message'</code> which soft-deletes the reported message and broadcasts the deletion.

`formatMessage(msg)` normalises every outgoing message: fixes the channel name (also handles a legacy channel id), and replaces deleted content with a placeholder.

9.3 Client side

- `utils/socket.js` — **singleton** `getSocket(token, onStatusChange)`: creates one connection with `auth: {token}`, reconnection (15 attempts, 1 s delay), transports websocket → polling. Reports status: connected / disconnected / reconnecting. `disconnectSocket()` on logout.
- `pages/Community.jsx` — loads channels, then messages for the active channel; registers socket listeners `message:receive`, `message:update`, `message:delete`, `message:react`, `typing:start`, `typing:stop` and removes them in the effect cleanup.
- `handleReceive` de-duplicates by `_id` (the sender gets the message both from the REST response and the broadcast). A message for another channel increments an **unread counter** instead.
- `activeChannelRef` (a `useRef`) is used inside listeners so they always see the current channel without re-subscribing — avoids the stale-closure problem.
- Components: `CommunityLayout` (mobile: list ↔ chat switching), `Channellist` (icons + unread badges), `ChatWindow` (header + connection badge), `MessageList` (date separators, auto-scroll to bottom, jump button), `MessageItem` (bubble, colour per sender, reply preview, reactions, edit / delete / copy / report menu), `MessageInput` (auto-growing textarea max 120 px, 2000-char limit, Enter to send / Shift+Enter newline, emoji picker, typing events debounced 2 s, read-only badge in Announcements), `ReportModal`.
- Admin `Community.jsx`: post announcements, delete any message, review reports (Reviewed / Dismissed / Action Taken + delete).

Say it like this: "I write messages through REST and push them through WebSockets. REST gives me validation, authorization and persistence in one place; Socket.IO rooms then broadcast only to users viewing that channel. The client de-duplicates by message id."

10. Uploads, Notices and Staff

10.1 Upload requests (uploadRoutes.js)

Admin creates a request (e.g. "Submit fee receipt by 30 Sept"); each student uploads a file; admin sees who submitted and who is pending.

Function / endpoint	Explanation
<code>getUserAccessDate(user)</code>	The student's <code>createdAt</code> (or <code>admissionDate</code>).
<code>isAdmittedUser(user)</code> (exported, tested)	True if the student has a valid access date.
<code>isUploadVisibleToUser(upload, user)</code> (exported, tested)	A student only sees requests created on or after the day he joined — a new student is not shown old requests.
<code>getUserLookupQuery(tokenUser)</code>	Builds <code>\$or</code> of <code>_id</code> / <code>username</code> / <code>email</code> to find the user.
GET /	Public: latest 10 requests. Logged-in: only visible ones, each with <code>submitted</code> flag and own submission, plus <code>pendingCount</code> (shown on the home page widget).
POST / (admin)	Create request: title, description, <code>dueDate</code> .
GET /all (admin)	All requests with submissions.
GET /:id	Student: request + only own submission. Admin: <code>studentStatus.submitted[]</code> , <code>pending[]</code> , <code>others[]</code> and a summary count.
POST /:id/submit (auth, file ≤10 MB)	Uploads to Cloudinary <code>hms_uploads</code> (deletes the previous file of this student first) or saves locally with a sanitised filename; replaces the student's previous submission in the array.

10.2 Notices (noticeRoutes.js)

- GET `/api/notices?limit=6` — public; newest notices plus `recentCount` = notices from the last 3 days (drives the **NEW** badge).
- GET `/all?page=&limit=` — admin, **skip/limit pagination** with total pages.
- POST / PUT / DELETE — admin CRUD with severity low / medium / high. PUT uses `runValidators: true` so max length still applies on update.
- Frontend `NoticeBoardSection` shows them on the home page; admin `NoticeBoard.jsx` has a create/edit modal.

10.3 Staff (staffRoutes.js)

- GET `/api/staff` public (home page Staff section with lightbox gallery: prev/next wrap-around using `modulo`).
- POST / PUT / DELETE admin, with image upload to Cloudinary `hms_staff`; on update or delete the old image is destroyed using the stored `publicId`.

11. AI Assistant — MCP Server + LLM (mcp_server)

The admin can ask questions in natural language ("How many students are in room 14?", "Who was absent yesterday?"). This is a separate **Python** service so the Node API stays simple and the AI part can be scaled or turned off independently.

11.1 What is MCP?

The **Model Context Protocol** is an open standard for exposing *tools* (functions with typed arguments and descriptions) to an LLM. `server.py` registers functions with `@mcp.tool()` using FastMCP; the same file can run as a stand-alone MCP server over stdio (`mcp.run(transport='stdio')`) so tools like Claude Desktop or Cursor can use it, **and** it is imported directly by `llm_service.py` for the web chatbot.

11.2 The 8 tools (server.py)

Tool	Purpose
<code>list_collections()</code>	Whitelisted collections with descriptions and estimated counts.
<code>describe_schema(collection)</code>	Allowed fields per collection — helps the LLM build correct queries.
<code>find_documents(collection, filter, projection, sort, limit)</code>	Safe find with validation, projection, sort, limit ≤ 50.
<code>count_documents(collection, filter)</code>	Safe count.
<code>aggregate_documents(collection, pipeline, limit)</code>	Read-only aggregation; only <code>\$match</code> <code>\$project</code> <code>\$sort</code> <code>\$limit</code> <code>\$group</code> <code>\$unwind</code> <code>\$count</code> allowed; <code>maxTimeMS=10000</code> .
<code>search_database(query, collections)</code>	Keyword search (escaped regex) across users, notices, staff and leaves.
<code>get_daily_attendance(date, status, student_username)</code>	Uses IST date; if today has no record it uses the latest date. Returns status counts, a filtered list, or one student's record via <code>\$unwind</code> pipelines.
<code>get_database_stats()</code>	Total students, rooms, notices, pending/approved leaves, staff, breakdown by department and year.

11.3 Safety layer — the most important part to explain

- **Collection whitelist** (`COLLECTIONS`) + aliases (`student` → `users`, `leave` → `leaveapplications`). Unknown collection → error. `resolve_collection_name()`.
- **Sensitive field redaction**: regex patterns for password, token, email, phone, mobile, aadhaar, bank, account, ifsc, address, and every URL field. `build_safe_projection()` removes them from query results, and `redact_sensitive_values()` masks any that slip through as `[REDACTED]`.
- **Operator whitelist** (`validate_query_filter`): only `$eq` `$ne` `$gt` `$gte` `$lt` `$lte` `$in` `$nin` `$regex` `$options` `$and` `$or` `$nor` `$exists` `$elemMatch` `$size`. Anything like `$where` (runs JavaScript) is rejected. Filtering on sensitive fields is also rejected, so the LLM cannot "guess" a phone number by filtering.
- **Read-only**: there are no insert/update/delete tools at all.
- **Limits**: `clamp_limit()` caps results at 50; aggregation time-out 10 s.
- `coerce_object_ids()` converts 24-hex strings in `_id/...Id` fields to ObjectId; `normalize_filter_types()` matches `roomNumber: "14"` against both string and int; `serialize()` converts ObjectId/datetime to JSON.

11.4 LLM orchestration (llm_service.py)

1. `ask_question(question, session_id, history)` is the entry point. If `GROQ_API_KEY` exists it runs the LLM pipeline, otherwise (or on error) the deterministic fallback.
2. `execute_llm_mcp_pipeline` builds a **system prompt** with today's IST date, the schema of each collection and rules (always use tools for data, never invent data).
3. Adds conversation context: last 8 messages from the client, or last 4 turns from the JSON memory file for that session.
4. `get_mcp_tool_definitions()` describes the tools in OpenAI-compatible JSON schema. `call_llm()` POSTs to Groq (`llama-3.3-70b-versatile`, fallback `llama-3.1-8b-instant`, temperature 0.1 for factual answers, 35 s timeout).
5. If the model returns `tool_calls`, each tool is executed with `mcp_server.execute_tool(name, args)` and the JSON result is appended as a `role: 'tool'` message. The loop repeats (max **5 rounds**) until the model answers in plain text.
6. `extract_text_tool_calls()` handles models that write tool calls as XML-like text `<find_documents>{...}</find_documents>`.
7. `remember_interaction()` stores the Q&A in `.conversation_memory.json` (last 12 turns per session).
8. `local_deterministic_fallback()` — keyword rules ("by department", "stats", "room 14", "absent", "attendance", "notice", "leave", "staff") call tools directly, so the assistant still works without the LLM.

11.5 api.py and the admin UI

- FastAPI with CORS; Pydantic models `QueryRequest` (question, session_id, history) and `QueryResponse` (answer, source, tools_used, data, errors).
- Endpoints: `GET /` and `/health` (status, collections, whether Groq is enabled), `GET /collections`, `POST /ask`.
- Admin `HostelNexus.jsx` (full page) and `HostelNexusWidget.jsx` (draggable floating chat on the dashboard): create a session id stored in localStorage, check `/health` every 15 s, send the last messages as history to `/ask`.

Say it like this: "The LLM never touches the database directly. It can only call eight read-only tools, and every tool validates the collection, the operators and the fields, redacts personal data and limits results. So even a prompt-injection cannot delete data or leak Aadhaar or bank numbers."

12. Student Portal (frontend)

12.1 App shell — App.jsx

Piece	Explanation
State: <code>token</code> , <code>user</code> , <code>profileImage</code>	Initialised lazily from <code>localStorage</code> (<code>useState(() => ...)</code>) so the user stays logged in after refresh.
<code>darkMode</code>	Read from <code>localStorage</code> or the OS setting <code>prefers-color-scheme</code> ; applied as <code>data-theme</code> on <code><html></code> ; CSS variables switch colours.
<code>handleLogin(user, token, img)</code>	Saves to state and <code>localStorage</code> (<code>useCallback</code> to keep a stable reference).
<code>handleLogout()</code>	Clears state and storage.
<code>handleProfileUpdate(updated User, imageUrl)</code>	Merges profile changes into state + storage.
Routes	<code>/</code> , <code>/about</code> , <code>/facilities</code> , <code>/gallery</code> , <code>/admission</code> , <code>/contact</code> , <code>/login</code> (redirects to <code>/profile</code> if logged in), <code>/signup</code> → <code>/login</code> (students are created by admin only), <code>/profile</code> (protected), <code>/community</code> , <code>/complaints</code> , <code>/uploads</code> .
Protected route pattern	<code>user ? <Profile/> : <Navigate to="/login" replace/></code> .

12.2 Pages

Page	What it does / key functions
Home.jsx	Composes sections: Hero, UploadsWidget, NoticeBoard, Features, About, Facilities, FoodAllowance, Events, Gallery, Staff, HostelRules, Admission, Documents, HostelJourney, FAQ, Contact, BackToTop.
Login.jsx	<code>validate()</code> → <code>handleSubmit()</code> POST <code>/api/user/login</code> , stores token/user, calls <code>onLogin</code> . Forgot-password mode: <code>handleForgotPassword()</code> does step 1 (send OTP) then step 2 (reset).
Profile.jsx (~970 lines)	Student dashboard. Loads profile, documents, leave history and attendance in effects with a <code>cancelled</code> flag (prevents <code>setState</code> after unmount). Features: edit profile with <code>validate()</code> (email & phone regex, required dept/year), photo compressed then uploaded (<code>handleImageUpload</code> , <code>handleSave</code> as <code>FormData</code>), change password with OTP (<code>handleSendOtp</code> , <code>handleVerifyOtp</code>), 8-document vault with in-app preview (<code>getPreviewType</code> → image / pdf / office / text), leave modal (<code>handleLeaveSubmit</code> , attachment required and compressed), attendance heatmap (<code>heatmapWeeks</code> pads the first week with nulls by <code>dayOfWeek</code> and groups days into 7-day columns, like GitHub), history grouped by month, year selector (last 5 years).
ComplaintBox.jsx	See section 8.
Community.jsx	See section 9.
Uploads.jsx + UploadsWidget.jsx	Lists admin upload requests with submitted / pending state; widget lets a student choose a file and submit per request (<code>handleFileChange</code> , <code>handleSubmit</code>).
About / Facilities / Gallery / Admission / Contact	Mostly static informational pages built from section components.

12.3 Reusable pieces

- `Navbar` — sticky header that changes style after 18 px scroll, mobile hamburger, shows Complaint and Community links **only when logged in**, dark-mode toggle, profile pill and logout.
- `useInView` hook — `IntersectionObserver` that fires once when a section scrolls into view (used for reveal animations); disconnects on cleanup.
- `compressImage.js` — wraps `browser-image-compression` (0.8 MB, 2400 px, web worker, quality 0.75). Returns the original file if compression fails or does not shrink it.
- `casteData.js` — dropdown lists for caste category and caste.
- `InputField`, `Button`, `ProfileCard`, `Footer`, `BackToTop` — small presentational components.

13. Admin Dashboard (admin)

13.1 App.jsx — role-based routing

- `getValidToken()` treats the strings 'null' / 'undefined' as no token (a common localStorage bug).
- `ProtectedRoute({allowedRoles})`: no token → login; role not allowed → alert + redirect (attendance takers go to `/attendance`).
- Admin-only: dashboard, users, users/:id, add-user, profile, notices, uploads, uploads/:id, staff, community, complaints, leaves, admin-users, hostel-nexus. Admin **and** attendance_taker: attendance, attendance-visuals.
- Client-side checks are only for UX — **the server re-checks every request**, so editing localStorage does not grant access.

13.2 Pages

Page	Explanation
Login.jsx / Register.jsx	Login stores <code>adminToken</code> , <code>adminUsername</code> , <code>adminRole</code> . Register is a 2-step form (details → OTP from root admin). Forgot/reset password with OTP.
Dashboard.jsx	<code>Promise.all</code> loads users, rooms-overview, leave notifications and absence alerts in parallel. Shows KPIs, room occupancy grid (click a room → modal with its students via <code>handleRoomClick</code>), notifications feed, caste / caste-category counts (switchable <code>countsGroupBy</code> ; clicking a bar opens Users filtered by <code>?groupBy=&value=</code>), recent students, and the floating AI widget.
Users.jsx	Student directory. Reads filter from URL query params; free-text search over name, username, college, stream, phones, room; groups by room and sorts room keys numerically (<code>localeCompare</code> with <code>numeric:true</code> , 'Unassigned' last); delete with confirm; click row → profile.
AddUser.jsx	Loads rooms-overview to show free beds; <code>handleCreate</code> → <code>/add-user</code> . Inline edit of email / roll / room (<code>handleUserUpdate</code> calls the right endpoint; handles the 409 confirmation for room change).
UserProfile.jsx	Full student profile, documents and the same attendance heatmap as the student sees.
Attendance.jsx / AttendanceVisuals.jsx	See section 6. Visuals draws an SVG line chart manually (polyline + points with tooltips), shows unapproved absentees for a date and a custom range report.
LeaveManagement.jsx	All leaves, attachment preview (image/PDF), comeback prompt → PATCH <code>/:id/comeback</code> .
NoticeBoard.jsx	CRUD modal, NEW badge for recent notices, handles 401 by logging out.
Uploads.jsx / UploadDetails.jsx	Create upload requests; view submitted vs pending students; preview files.
StaffManagement.jsx	Staff CRUD with photo.
Complaints.jsx	See section 8.
Community.jsx	Announcements, moderation, reports.
AdminUsers.jsx	List admins with last login; delete with root-admin OTP.
HostelNexus.jsx + widget	AI chat (section 11).
Sidebar.jsx	Navigation filtered by role, theme toggle (stored as <code>adminTheme</code>), badges refreshed every 10 s, logout.

14. End-to-End Flows (explain these on a whiteboard)

Flow A — Student onboarding and login

1. Admin → Add User: username, temporary password, room. Server checks room capacity and hashes the password.
2. Admin gives credentials to the student. Student logs in on the portal → `/api/user/login` → JWT (7 days).
3. Student completes profile (personal, address, caste, bank, college) and uploads a photo (compressed, then Cloudinary).
4. Student uploads 8 verification documents into the document vault.
5. Forgotten password → OTP to registered email → reset.

Flow B — Leave to comeback

1. Student submits leave with dates, reason and proof (compressed image/PDF).
2. Server: date validation → overlap check → upload proof → save (Approved) → mark those dates Absent (`leave-auto`).
3. At night, attendance taker marks the room; the on-leave student is forced Absent regardless of the checkbox.
4. If the end date passes without return, `/leaves/notifications` raises an overdue alert on the dashboard.
5. When the student reports back, admin clicks Comeback → future auto-absents are removed.
6. 5 days after completion, the record is auto-cleaned.

Flow C — Complaint

1. Student files complaint with category, priority, optional photo.
2. Admin sidebar badge updates within 10 s; complaints page refreshes every 12 s.
3. Admin responds → status auto-moves to In Progress; later marks Resolved (timestamp stored).
4. Student sees status and the admin's reply under My complaints.

Flow D — Chat message

1. On opening Community the client connects Socket.IO with the JWT and emits `community:join` for the channel.
2. User sends text → REST POST saves the message → server emits `message:receive` to the channel room.
3. Every client in that room appends it (dedupe by id); clients in the other channel get an unread badge.

Flow E — AI question

1. Admin types "How many students are absent today?" in HostelNexus → POST `http://127.0.0.1:8000/ask`.
2. LLM reads the system prompt + tool list and returns a tool call `get_daily_attendance(status='Absent')`.
3. Python runs the safe tool against MongoDB (IST date) and sends the JSON back to the LLM.
4. LLM writes the final answer with the exact number and names; response includes `tools_used` for transparency.

15. Security, Performance and Testing

Security measures implemented

- Passwords hashed with **bcrypt** (salt rounds 10); never returned in responses.
- **JWT** auth with role claims, 7-day expiry; admin tokens re-checked against the DB (disabled admins blocked).
- **OTP** for password reset, password change, admin registration and admin deletion; stored only as SHA-256 hash, expires in 5 min, single use.
- Admin registration and deletion need approval of the **root admin** mailbox.
- Input validation on server (required fields, email regex, date format, room range, length limits, enum values) and on client.
- **Whitelisting** of profile fields so mass-assignment of **roomNumber/password** is impossible from the profile form.
- File size limits with Multer; images compressed client-side.
- Chat: announcement channel enforced on the server; only owner/admin can edit/delete; moderation reports.
- Socket connections authenticated during handshake.
- AI layer: read-only tools, operator and field whitelists, PII redaction, result limits and time-outs.
- Unique DB indexes (username, attendance date, document userId) guard against duplicates.

Performance techniques

- **Promise.all** for independent queries (dashboard, absence rows, stats).
- **.lean()** on read-only queries (plain JS objects, faster than Mongoose documents).
- **.select()** to fetch only needed fields.
- Maps for O(1) lookups when joining attendance with users in memory.
- Indexes: **{channel, createdAt}** on messages, **date** on attendance.
- Pagination: notices (skip/limit) and chat (cursor **before**).
- Client image compression; Cloudinary **quality:auto** and **fetch_format:auto** (serves WebP/AVIF when supported).
- Socket.IO rooms so events only go to interested clients.

Testing

Backend unit tests use Node's built-in test runner (**node:test** + **node:assert/strict**), no extra library. Run from **backend/** with **node --test tests/**.

Test file	What it checks
passwordOtp.test.js	OTP generation / verification / expiry.
passwordReset.test.js	Reset payload validation rules.
roomNumber.test.js	Room number normalisation.
attendanceAbsentsQuery.test.js	resolveAbsentsDateRange for a single date and for a month.
leaveApplication.test.js	LeaveApplication schema defaults.
documentSchema.test.js	Document schema fields / uniqueness.
uploadRoutes.test.js	isAdmittedUser and isUploadVisibleToUser date logic.

Test file	What it checks
mcp_server/test_suite.py	Integration script: runs every MCP tool against the real DB and a few LLM questions.

Pure logic was pulled into small exported functions precisely so it can be tested without a database — a good point to mention.

16. Honest Limitations and Future Improvements

Interviewers often ask **"What would you improve?"** or spot a weakness in the code. Knowing these shows maturity. Say: **"I am aware of it, here is how I would fix it."**

Limitation in current code	How to fix
OTPs are kept in in-memory <code>Maps</code> → lost on server restart and do not work with multiple server instances.	Store in Redis with TTL (or a MongoDB collection with a TTL index).
<code>DELETE /api/admin/users/:id</code> and <code>GET /users/:id/documents</code> only require any valid token, and leave <code>/all, /:id/status, /:id/comeback, /notifications</code> plus <code>GET /complaints/:id</code> have no auth.	Add <code>adminAuth</code> to these routes; add ownership checks (student may only read own data).
JWT secret has a hard-coded fallback <code>'your_super_secret_key_here'</code> .	Fail at startup if <code>JWT_SECRET</code> is missing.
Auth middleware is copy-pasted in 6 route files with slightly different rules.	Use the single shared middleware file everywhere (DRY).
Token in <code>localStorage</code> is readable by any injected script (XSS).	<code>HttpOnly</code> , <code>Secure</code> , <code>SameSite</code> cookies + <code>CSRF</code> token; short access token + refresh token.
<code>Math.random()</code> for OTP is not cryptographically secure.	<code>crypto.randomInt(100000, 1000000)</code> .
No rate limiting on login / OTP endpoints (brute force).	<code>express-rate-limit</code> , account lockout after N failures.
Complaint search builds <code>new RegExp(userInput)</code> without escaping (ReDoS / regex injection).	Escape special characters before building the regex.
Leaves are auto-approved on submit, although the model default and admin UI support Pending → Approved.	Make auto-approve a config flag; default to Pending for warden review.
Attendance window message says 8 PM but code uses 9 PM; 20 rooms hard-coded in attendance routes while user routes use <code>ROOM_OVERVIEW</code> .	Single config source for window times and rooms.
Student attendance endpoints load all attendance documents then filter in JS.	Query <code>{'students.username': u}</code> with an index / date range, or aggregation.
<code>tempPassword</code> is set on the User but not in the schema (Mongoose strict mode drops it) and returned in the API response.	Force password change on first login instead of exposing it.
<code>cors()</code> allows every origin; Socket.IO <code>origin: '*'</code> .	Restrict to the two frontend domains.
Polling (10–12 s) for complaint counts.	Reuse the existing Socket.IO connection to push updates.
No automated frontend or API integration tests.	Jest + React Testing Library; Supertest with an in-memory MongoDB.
README advertises 24 MCP tools; code has 8 general tools.	Update docs (the 8 generic tools cover those use-cases).

Feature ideas

- Mess / food-allowance billing and payment gateway.
- QR or face-recognition attendance, geo-fencing.
- Push / SMS notifications to parents when a student is absent or overdue from leave.
- Document verification workflow (Verified / Rejected with remarks).
- Docker Compose for one-command setup; CI pipeline running tests.

17. Interview Questions and Answers

General / architecture

Q. Why did you split frontend and admin into two apps?

Different users, different security and different bundles. Students never download admin code, and each app can be deployed and scaled independently. Both share the same API.

Q. Why MongoDB instead of MySQL?

Student profiles have many optional fields and evolved often; a document model let me add fields without migrations. Attendance fits naturally as one document per day with an embedded array. For heavy relational reporting I would consider SQL, but for this scale MongoDB with indexes and aggregation works well.

Q. What is the difference between embedding and referencing? Where did you use each?

Embedding stores related data inside the same document (attendance students array, upload submissions, message reactions) — one read, atomic update. Referencing stores an ObjectId and uses `populate()` (leave → user, complaint → user, message → sender) — avoids duplication when data is shared or large.

Q. Why HashRouter?

The apps are deployed as static files. With BrowserRouter, refreshing `/profile` asks the server for that path and gets 404 unless rewrite rules exist. HashRouter keeps the route after `#`, which the server never sees.

Q. How does the Vite proxy help?

In development the React app runs on 3000 and the API on 5000 — different origins, so CORS applies. The proxy forwards `/api` and `/socket.io` from the dev server to 5000, so the browser sees one origin.

Q. What happens if port 5000 is busy?

`startServer` catches `EADDRINUSE` and retries on the next port, up to five times.

Authentication and security

Q. Explain JWT. Where is it stored and how is it verified?

A JWT has header, payload and signature, base64-encoded. The server signs `{userId, username, role}` with a secret and 7-day expiry. The client keeps it in `localStorage` and sends `Authorization: Bearer`. Middleware calls `jwt.verify`; if the signature or expiry is wrong it returns 401. No session is stored on the server.

Q. How do you log a user out with JWT?

The client deletes the token. Because JWT is stateless the token remains valid until expiry — to revoke immediately you need a deny-list or short-lived access tokens with refresh tokens. For admins I partly solve this by checking `isActive` in the DB on every request.

Q. Why bcrypt and not SHA-256 for passwords?

bcrypt is deliberately slow and salted, so brute-force and rainbow-table attacks are expensive. SHA-256 is fast, which is good for OTP hashing (short-lived) but bad for passwords.

Q. How does your OTP flow work?

Generate 6 digits, email the plain OTP with SendGrid, store only its SHA-256 hash with an expiry of 5 minutes in a Map keyed by username. On verify, compare hashes and check expiry, then delete the entry so it cannot be reused.

Q. How is role-based access implemented?

Role is in the JWT. Backend middleware (`adminAuth`, `attendanceAuth`) checks it and, for admin routes, reloads the admin from the DB. Frontend `ProtectedRoute` hides pages by role for UX, but the backend is the real gate.

Q. How do you prevent a student from changing his room number via the profile API?

The profile route copies only a whitelist of 21 allowed fields from `req.body`; `roomNumber` and `password` are not in the list.

Q. What is the difference between 401 and 403?

401 = not authenticated (no/invalid token). 403 = authenticated but not permitted.

Backend logic

Q. How do you stop a room from being over-allocated?

Parse room capacities from `ROOM_OVERVIEW`, count current occupants with `countDocuments({roomNumber})`, reject if `count ≥ capacity`. For room change I exclude the student himself from the count.

Q. Explain the attendance time window and locking.

Marking is allowed only for today and only between 21:00 and 24:00 IST, computed with the Asia/Kolkata time zone so the server's own zone does not matter. Outside the window any existing record is locked. This prevents backdating or editing history.

Q. How do you save one room without overwriting other rooms?

Read the day document, filter out entries of that room, append the new entries, then `findOneAndUpdate({date}, {$set: {students}}, {upsert:true}).$setOnInsert` keeps `firstSavedAt` from the first save.

Q. There is a race condition if two takers save different rooms at the same moment — how would you fix it?

Use atomic array operators instead of read-modify-write: `$pull` the room's entries then `$push` the new ones, or keep a version field and retry on conflict (optimistic locking), or store one document per room per day.

Q. How do leave and attendance interact?

Approving leave writes Absent entries marked `leave-auto` for every leave date. When marking attendance, students on approved leave are forced Absent. Absence alerts exclude dates covered by leave. Comeback removes future auto-entries.

Q. How do you detect overlapping leaves?

Two ranges overlap when `existing.start ≤ new.end AND existing.end ≥ new.start`. I query Pending/Approved leaves of the same user with exactly that condition.

Q. How is the Excel report generated and sent?

ExcelJS builds a workbook in memory: merged styled title, header row, one row per student, P/A coloured, frozen panes and auto-filter. I set the `xlsx` Content-Type and `Content-Disposition: attachment` and stream it with `workbook.xlsx.write(res)`.

Q. How do file uploads work?

Multer with `memoryStorage` gives `req.file.buffer`. I convert the buffer to a stream with `streamifier` and pipe it into `cloudinary.uploader.upload_stream`. I store the returned `secure_url` (and sometimes `public_id`). When replacing, I extract the `public_id` from the old URL and destroy it. If Cloudinary is not configured, some modules fall back to local disk under `/uploads`.

Q. Why did you define `/users/counts` before `/users/:id`?

Express matches routes in order; otherwise "counts" would be captured as the `:id` parameter.

Q. What is `.lean()`?

It returns plain JavaScript objects instead of full Mongoose documents — faster and lighter, used when I only read data.

Q. What does `populate()` do?

It replaces an ObjectId reference with the referenced document (like a join), e.g. `leave.userId` → `{username, fullName, roomNumber}`.

Q. Explain an aggregation you wrote.

`/users/counts: [{ $group: { _id: '$casteCategory', count: { $sum: 1 } } }, { $sort: { count: -1 } }]` — counts students per category for the dashboard chart.

Real-time

Q. WebSocket vs HTTP polling?

Polling asks the server repeatedly and wastes requests; WebSocket keeps one open connection so the server can push instantly. Socket.IO adds rooms, reconnection and a polling fallback.

Q. How do you authenticate a socket?

The client passes the JWT in `auth.token`; an `io.use` middleware verifies it before the connection is accepted and attaches the user to `socket.user`.

Q. How do typing indicators work?

On input the client emits `typing:start` and sets a 2-second timer that emits `typing:stop`. The server relays to the room with `socket.to(room)`, which excludes the sender.

Q. Why soft delete messages?

Replies and reports still reference the message, and moderators need an audit trail. The content is replaced with a notice and `isDeleted` is set.

Q. How do you avoid showing a message twice?

The sender receives the message in the REST response and again via broadcast, so the client checks if a message with the same `_id` already exists before appending.

React

Q. Why `useRef` for the active channel inside socket listeners?

Listeners are registered once; a normal state variable would be captured with its old value (stale closure). A ref always points to the current value without re-registering listeners.

Q. Why the `cancelled` flag in `useEffect` fetches?

If the component unmounts before the fetch finishes, setting state would warn / leak. The cleanup sets `cancelled = true` and the async function checks it.

Q. Why lazy initial state `useState(() => localStorage.getItem(...))`?

The function runs only on the first render, so `localStorage` is not read on every re-render.

Q. What is `useMemo` / `useCallback` used for here?

`useMemo` caches derived data such as filtered complaint lists; `useCallback` keeps handler identity stable (`handleLogin`) so children/effects do not re-run unnecessarily.

Q. How is the attendance heatmap built?

The API returns one entry per day with `dayOfWeek`. The frontend pads the first week with empty cells up to the first day's weekday, then slices the days into columns of 7 and colours each cell by status.

AI / MCP

Q. What is MCP and why use it?

Model Context Protocol is a standard for exposing tools to LLMs. The same tool server works with my web chatbot and with any MCP client (Claude Desktop, Cursor) without rewriting.

Q. What is tool / function calling?

I send the LLM a JSON schema for each tool. Instead of answering, the model can return a tool name plus arguments. My code executes it, returns the result as a tool message, and the model continues until it writes a final answer.

Q. How do you stop the LLM from leaking or deleting data?

Only read tools exist; collection whitelist; operator whitelist (no `$where`); sensitive fields excluded from projection, blocked in filters and masked in output; result limit 50; aggregation timeout.

Q. What if Groq is down?

`call_llm` tries the 70B model then the 8B model; if both fail, `local_deterministic_fallback` answers common questions with keyword rules calling the same tools.

Q. Why temperature 0.1?

Low randomness gives consistent, factual answers — important when reporting numbers from a database.

Q. How is chat memory handled?

Each browser has a session id in localStorage. The client sends recent history; the server also keeps the last 12 turns per session in a JSON file.

Behavioural / project questions

Q. What was the hardest problem?

Good answers: keeping leave and attendance consistent (auto-absent, comeback correction, alerts excluding leave days), or making the AI safe (whitelists and redaction), or timezone bugs — solved by storing dates as 'YYYY-MM-DD' strings and computing the window in Asia/Kolkata.

Q. How did you test it?

Unit tests with `node:test` for the pure logic, the MCP integration script, and manual end-to-end testing of each role (student, admin, attendance taker) including edge cases like overlapping leave, full rooms and locked attendance.

Q. How would you deploy it?

Frontend and admin as static builds (`npm run build`) on a static host or Nginx; Express on a Node host / PM2 with environment variables; MongoDB Atlas; FastAPI with Uvicorn behind a reverse proxy; HTTPS everywhere and restricted CORS.

Q. How would it scale to 10 hostels?

Add a `hostelId` to every collection and to the JWT (multi-tenancy), move OTPs to Redis, use the Socket.IO Redis adapter for multiple API instances, add indexes on `hostelId + date`, and move heavy reports to background jobs.

18. Quick Revision Sheet

All API routes at a glance

Base path	Routes
/api/admin & /api (users)	POST add-user · PUT users/:id/email rollnumber room · POST register, register/verify · POST login · POST user/login · POST user/forgot-password, user/reset-password · POST forgot-password, reset-password · GET admin-users · POST admin-users/:id/request-delete-otp · DELETE admin-users/:id · GET/POST profile · POST profile/password/otp verify · GET users · GET rooms-overview · GET users/counts · GET/DELETE users/:id · GET/POST documents · GET users/:id/documents
/api/attendance	GET summary · GET visuals · GET absence-alerts · GET absents · POST range-report · GET/POST room/:roomNumber · GET student/my-attendance · GET student/:id/attendance · GET export
/api/leaves	POST submit · GET my · GET all · PATCH :id/status · PATCH :id/comeback · GET notifications
/api/complaints	POST submit · GET my · GET stats · GET all · GET :id · PATCH :id/respond · DELETE :id
/api/community	GET channels · GET/POST channels/:id/messages · PUT/DELETE messages/:id · POST messages/:id/react · POST messages/:id/report · GET admin/reports · PUT admin/reports/:id
/api/uploads	GET / · POST / · GET all · GET :id · POST :id/submit
/api/notices	GET / · GET all · POST / · PUT :id · DELETE :id
/api/staff	GET / · POST / · PUT :id · DELETE :id
AI :8000	GET / · GET health · GET collections · POST ask

Socket.IO events

Direction	Events
Client → Server	community:join, community:leave, typing:start, typing:stop
Server → Client	message:receive, message:update, message:delete, message:react, typing:start, typing:stop, user:online, user:offline

Environment variables

Service	Variables
backend	PORT, MONGODB_URI, JWT_SECRET, ADMIN_USERNAME, ADMIN_PASSWORD, ADMIN_ROOT_EMAIL, CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, SENDGRID_API_KEY, EMAIL_FROM, ROOM_OVERVIEW
frontend / admin	VITE_API_BASE_URL, VITE_SOCKET_URL (frontend), VITE_FASTAPI_URL (admin)
mcp_server	MDB_MCP_CONNECTION_STRING, DATABASE_NAME, GROQ_API_KEY, GROQ_MODEL, HOSTEL_API_HOST, HOSTEL_API_PORT

Run commands

```
cd backend    && npm install && npm run dev    # API + sockets on :5000
cd frontend   && npm install && npm run dev    # student portal on :3000
cd admin      && npm install && npm run dev    # admin panel on :5173
cd mcp_server && pip install -r requirements.txt && python api.py  # AI on :8000
cd backend    && node --test tests/          # unit tests
```

Say it like this: Walk through it in this order: problem → architecture diagram → one deep feature (attendance + leave) → security → AI assistant → what you would improve. Show the code for one route while you talk.